



Bahria University
Discovering Knowledge

SOFTWARE DESIGN AND ARCHITECTURE(SDA) LECTURE 8

ENGR.RAMSHA MASHOOD

SENIOR LECTURER

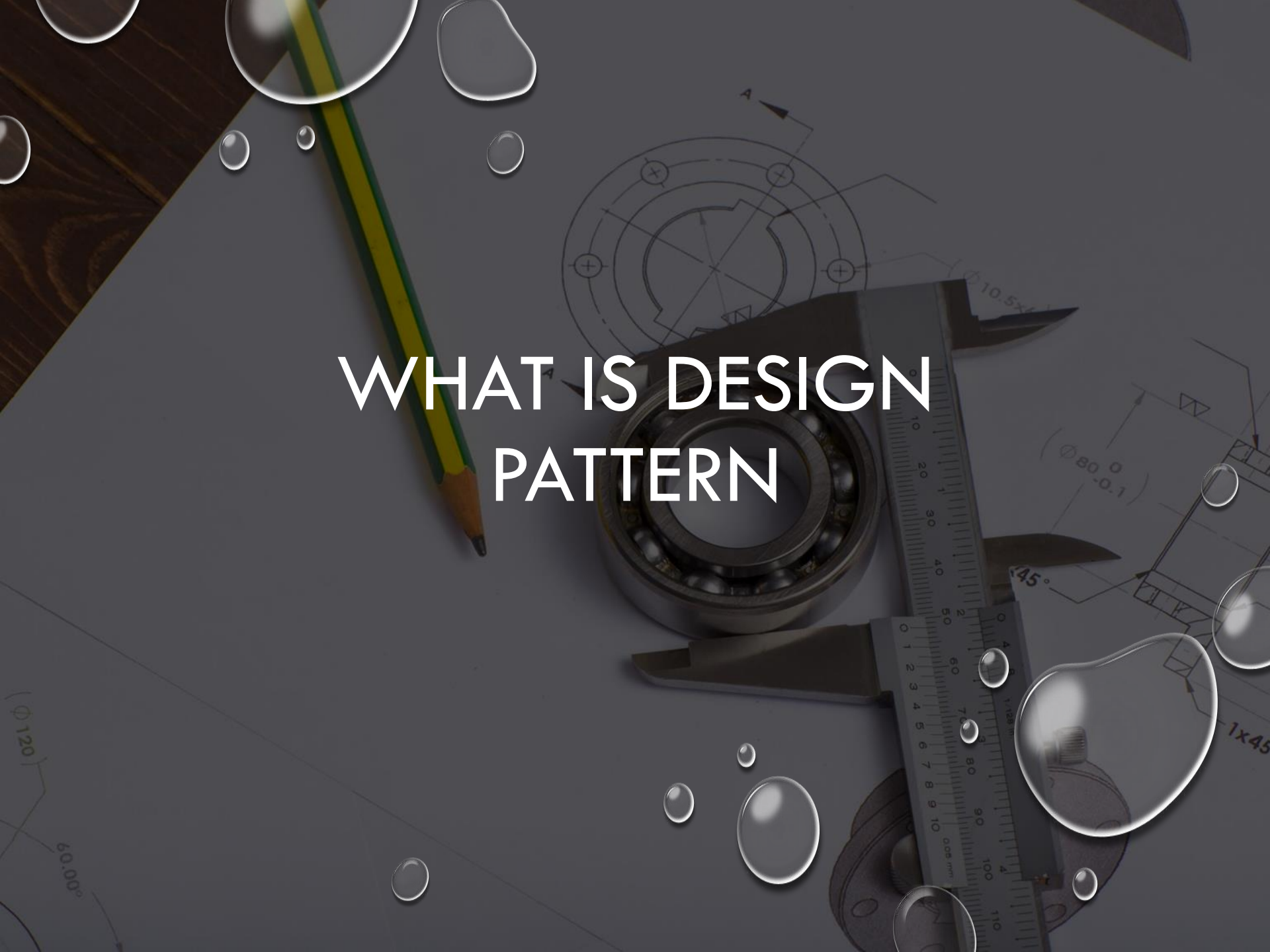
DEPARTMENT OF SOFTWARE ENGINEERING

BAHRIA UNIVERSITY KARACHI CAMPUS



BEHAVIORAL PATTERNS

WHAT IS DESIGN PATTERN

The background is a dark, textured surface featuring technical drawings. A green and yellow pencil lies diagonally on the left. A circular mechanical part, possibly a bearing, is centered. A vernier caliper is positioned on the right, with its jaws open. Several water droplets of varying sizes are scattered across the scene, adding a sense of freshness and precision. Technical annotations like 'Ø120', '60.00°', 'Ø10.5x', 'Ø80 0 -0.1', '45°', and '1x45' are visible on the drawings.

DEFINITION

EACH PATTERN DESCRIBES A PROBLEM WHICH OCCURS OVER AND OVER AGAIN, AND THEN DESCRIBES THE CORE OF THE SOLUTION TO THAT PROBLEM, IN SUCH A WAY THAT YOU CAN USE THAT SOLUTION MILLION TIMES OVER, WITHOUT EVER DOING IT THE SAME WAY TWICE.

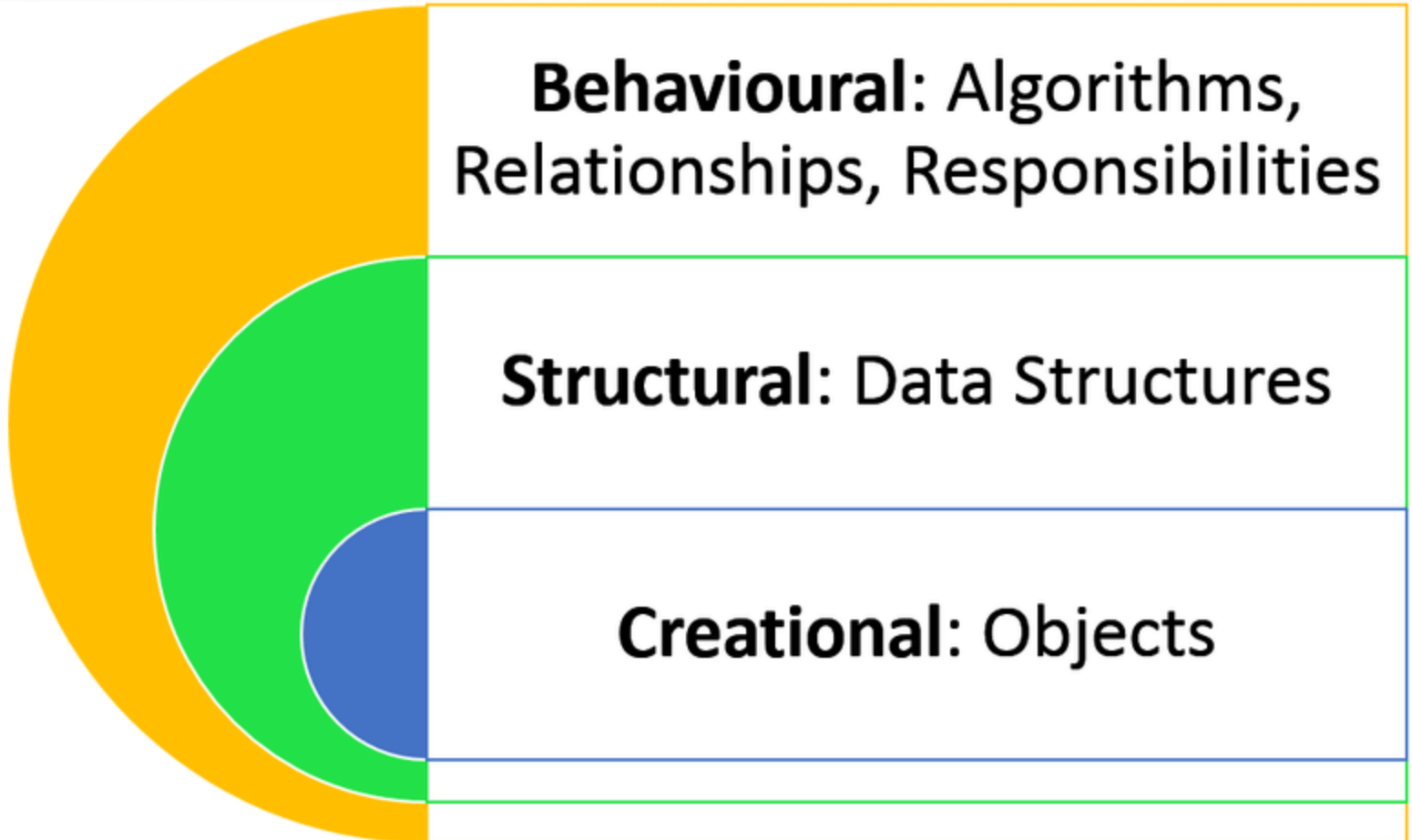
(CHRISTOPHER ALEXANDER)

(BOOK: DESIGN PATTERN BY ERICH GAMMA | RICHARD HELM)

DESIGN PATTERN

- In software engineering, a **design pattern** is a general **repeatable** solution to a commonly occurring problem in software design.
- **Design patterns** are **typical solutions** to **common problems** in software design.
- Each pattern is like a **blueprint** that you can customize to solve a particular design problem in your code.
- A design pattern isn't a finished design that can be transformed directly into code.
- It is a description or **template** for how to solve a problem that can be used in many different situations.
- *The 23 Gang of Four (GoF)* patterns are generally considered the foundation for all other patterns.

Design Patterns

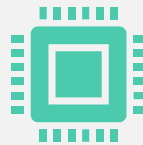


		<i>Purpose</i>		
		Creational	Structural	Behavioral
Scope	Class	Factory Method	Adapter (class)	Interpreter Template Method
	Object	Abstract Factory Builder Prototype Singleton	Adapter (object) Bridge Composite Decorator Flyweight Facade Proxy	Chain of Responsibility Command Iterator Mediator Memento Observer State Strategy Visitor



9

CLASSIFICATION OF DESIGN PATTERNS



Creational patterns provide object creation mechanisms that increase flexibility and reuse of existing code.

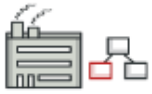


Structural patterns explain how to assemble objects and classes into larger structures, while keeping these structures flexible and efficient.

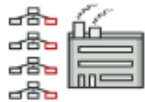


Behavioral patterns take care of effective communication and the assignment of responsibilities between objects.

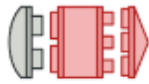
CLASSIFICATION



Factory Method



Abstract Factory



Adapter



Bridge



Chain of Responsibility



Command



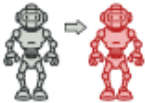
Iterator



Mediator



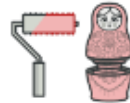
Builder



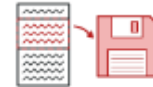
Prototype



Composite



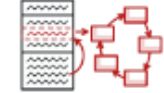
Decorator



Memento



Observer



State



Strategy



Singleton



Facade



Flyweight



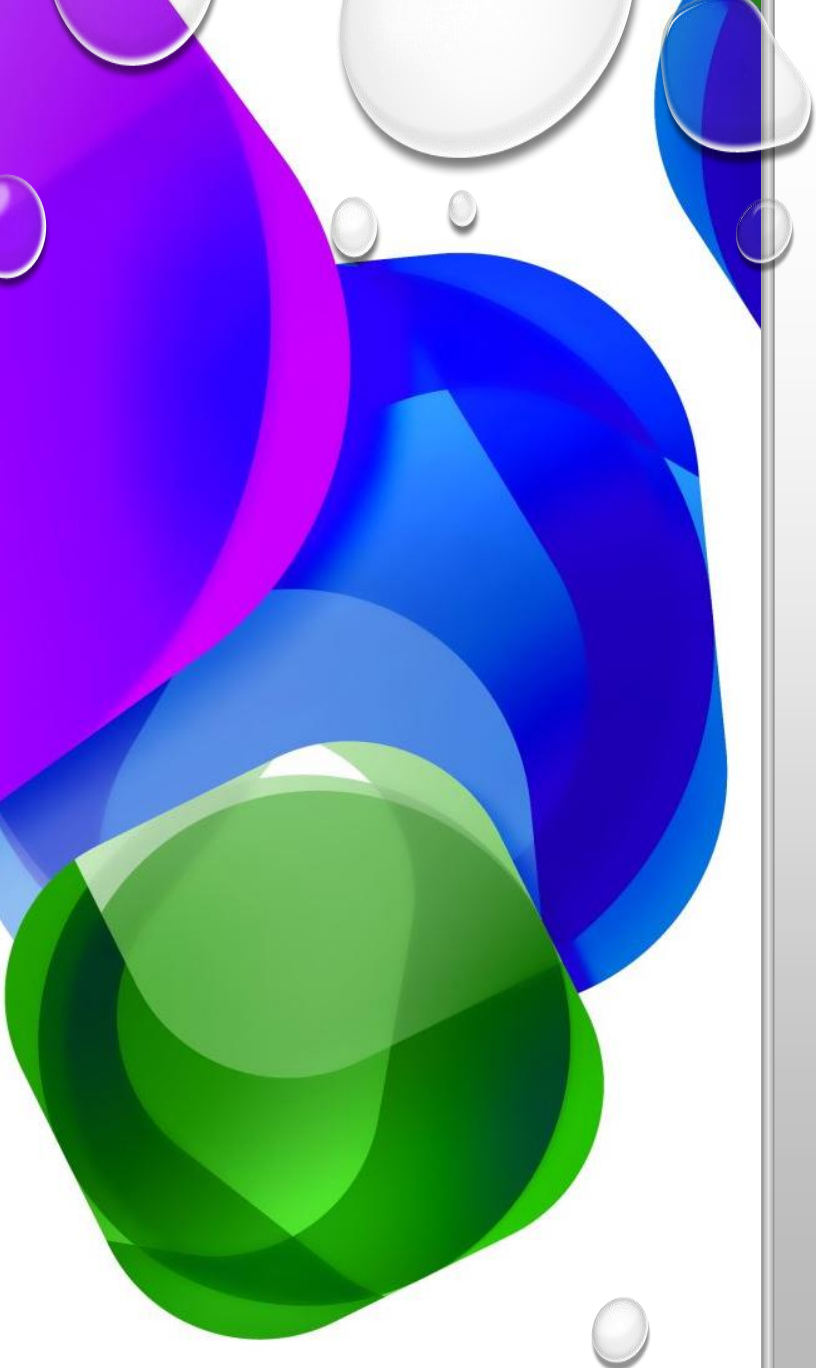
Template Method



Visitor



Proxy



BEHAVIORAL PATTERN

April 30, 2024

10



Behavioral pattern

- Behavioral patterns are concerned with *algorithms* and *communication* between them.
- The operations that make up a single algorithm might be *split* up between different classes, making a complex arrangement that is difficult to manage and maintain.
- The behavioral patterns capture ways of expressing the *division* of *operations* between classes and optimize how the communication should be handled.

Chain of Responsibility pattern

- The Chain of Responsibility pattern works with a list of **Handler** objects that have limitations on the nature of the requests they can deal with.
- If an object cannot handle a request, it passes it on to the next object in the chain.
- At the end of the chain, there can be either default or exceptional behavior.

OR

- *A way of passing a request between a chain of objects*
- This pattern defines a chain of processing objects in a chain in such a way that the incoming request is processed by each processing objects in sequence.

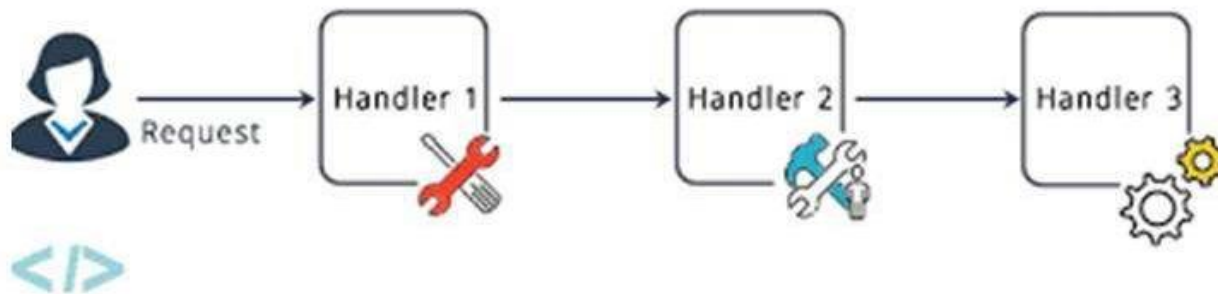
Chain of Responsibility pattern

- Chain of responsibility pattern is used to achieve **loose coupling** in software design where a request from the client is passed to a chain of objects to process them.
- Later, the object in the chain will decide themselves who will be processing the request and whether the request is required to be sent to the next object in the chain or not.
- This pattern is recommended when **multiple objects** can handle a request and the handler doesn't have to be a specific object.
- Also, the handler is determined at **runtime**.

***Tight coupling** means classes and objects are **dependent** on one another. In general, tight coupling is usually not good because it reduces the flexibility and re-usability of the code while **Loose coupling** means **reducing the dependencies** of a class that uses the different class directly.*

Chain of Responsibility Example

- Imagine members of the public (such as you and me) approaching a bank, a government office, a hospital, or a similar institution with a request.
- In the case of a bank, we know that a first level of clerks will handle straightforward requests.
- Those requests requiring more inspection will be sent up to a supervisor, and ultimately the manager might have to authorize or rule on a request.
- The clerk, supervisor, an manager together are an example of a chain of responsibility.



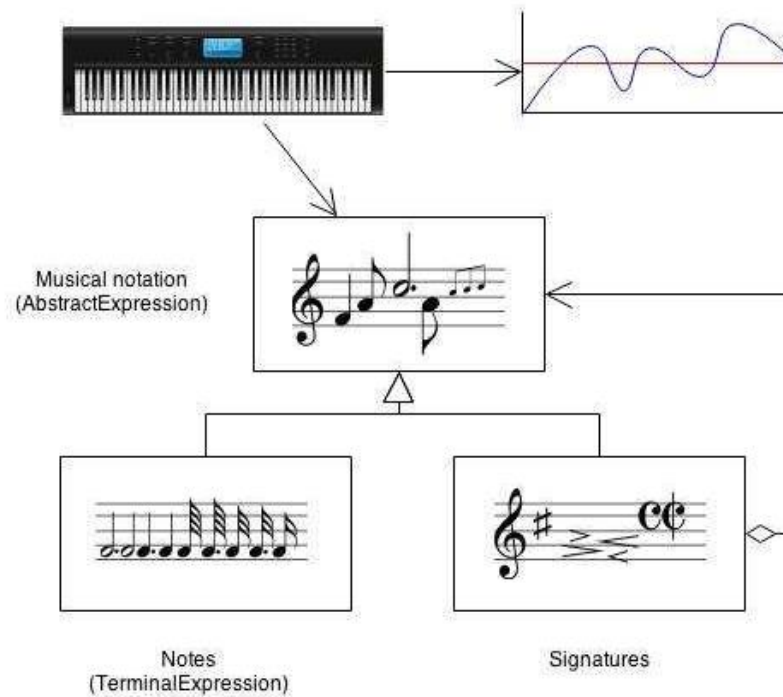
Interpreter pattern

- The *Interpreter* pattern defines a **grammatical representation** for a language and an interpreter to interpret the grammar.
- Interpreter pattern allows us to interpret grammar in to code solutions.
- The Interpreter pattern supports the interpretation of instructions written in a language or notation defined for a specific purpose.
- The notation is precise and can be defined in terms of a grammar.

Interpreter pattern

- Musicians are examples of *Interpreters*.
- The pitch of a sound and its duration can be represented in musical notation on a staff.
- This notation provides the language of music.
- Musicians playing the music from the score are able to reproduce the original pitch and duration of each sound represented.
- An example of this is the musical keyboard, which interprets clicks on keys for specific tones and notes.

Interpreter pattern Example



Strategy pattern

- A Strategy defines a set of algorithms that can be used interchangeably.
- The Strategy pattern involves removing an algorithm from its host class and putting it in a separate class.
- There may be different *algorithms (strategies)* that are applicable for a given problem.
- The Strategy pattern enables a client to choose which algorithm to use from a family of algorithms and gives it a simple way to access it.
- The algorithms can also be expressed independently of the data they are using.

Interpreter pattern

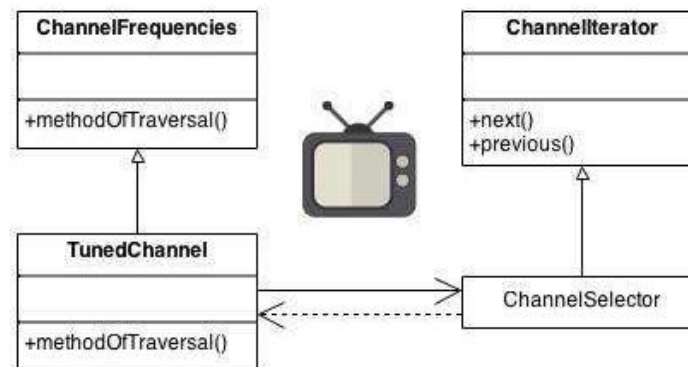
- Modes of transportation to an airport is an example of a *Strategy*.
- Imagine that you have to get to the university. You can catch a bus, order a cab, or get on your motorbike. These are your transportation strategies. You can pick one of the strategies depending on factors such as budget or time constraints.

Iterator pattern

- *Sequentially access the elements of a collection.*
- The Iterator provides ways to access elements of an *aggregate object sequentially* without exposing the underlying structure of the object.

Iterator pattern Example

- On early television sets, a dial was used to change channels. When channel surfing, the viewer was required to move the dial through each channel position, regardless of whether or not that channel had reception. On modern television sets, a next and previous button are used. When the viewer selects the "next" button, the next tuned channel will be displayed.



Command Design pattern

- *Encapsulate a command request as an object*
- In this pattern an object is used to represent and encapsulate all the information needed to call a method at a later time..

Command Design pattern

- Suppose you get to a nice restaurant and sit at the table by the window.
- A friendly waiter approaches you and quickly takes your order, writing it down on a piece of paper.
- The waiter goes to the kitchen and sticks the order on the wall.
- After a while, the order gets to the chef, who reads it and cooks the meal accordingly.
- The cook places the meal on a tray along with the order.
- The waiter discovers the tray, checks the order to make sure everything is as you wanted it, and brings everything to your table.
- ***The paper order serves as a command.***
- It remains in a queue until the chef is ready to serve it.
- The order contains all the relevant information required to cook the meal.
- It allows the chef to start cooking right away instead of running around clarifying the order details from you directly.

Command Design pattern

- Command pattern allows a request to exist as an object. Ok let's understand what it means. Consider the Example 'Menu and Commands' we have different actions depending on which menu is clicked. So depending on which menu is clicked we have passed a string which will have the action text in the action string. Depending on the action string we will execute the action.

Mediator pattern

- *Defines simplified communication between classes*
- In this pattern communication between objects is encapsulated with a mediator object.
- Objects no longer communicate directly with each other, but instead communicate through the mediator.

Mediator pattern Example

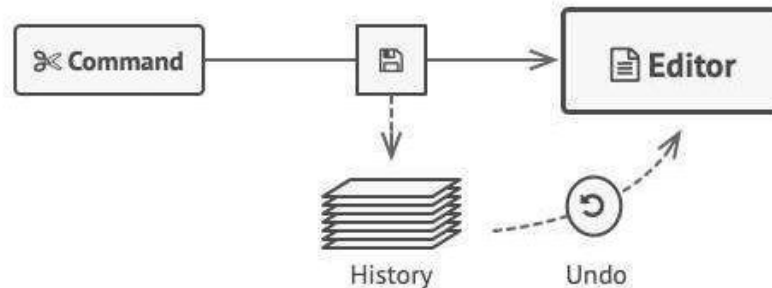
- Pilots of aircraft that approach or depart the airport control area don't communicate directly with each other. Instead, they speak to an air traffic controller, who sits in a tall tower somewhere near the airstrip. Without the air traffic controller, pilots would need to be aware of every plane in the vicinity of the airport, discussing landing priorities with a committee of dozens of other pilots. That would probably skyrocket the airplane crash statistics.
- The tower doesn't need to control the whole flight. It exists only to enforce constraints in the terminal area because the number of involved actors there might be overwhelming to a pilot.

Mementos pattern

- *Capture and restore an object's internal state*
- Mementos capture and externalize an object's internal state allowing the object to be restored to this state later.

Mementos pattern

- The *Memento* captures and externalizes an object's internal state, so the object can be restored to that state later.



Observer pattern

- *A way of notifying change to a number of classes.*
- ***Observer updates objects affected by a single object***
- An observable object called 'Subject' maintains a list of objects called 'Observers'. Subject notifies the observers of any state changes.

Observer pattern

- Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.
- Observer pattern helps us to communicate between parent class and its associated or dependent classes. There are two important concepts in observer pattern '**Subject**' and '**Observers**'. The subject sends notifications while observers receive notifications if they are registered with the subject. Below Example 'Subject and observers' shows how the application (subject) sends notification to all observers (email, event log and SMS).
- It is the pattern which communicates to the subscribed objects when there is a change in the subject.

Observer pattern

- Customers get notified by email, message in the mobile, hard copy of letter in postal address when the loan application is approved by the bank.

Observer pattern

- **Observer** design pattern that lets you define a subscription mechanism to notify multiple objects about any events that happen to the object they're observing.
- If you subscribe to a newspaper or magazine, you no longer need to go to the store to check if the next issue is available. Instead, the publisher sends new issues directly to your mailbox right after publication or even in advance.
- The publisher maintains a list of subscribers and knows which magazines they're interested in. Subscribers can leave the list at any time when they wish to stop the publisher sending new magazine issues to them.

State Design pattern

- *Alter an object's behavior when its state changes.*
- **OR**
- State pattern allows an object to alter its behavior when its internal state changes.

State Design pattern

- The buttons and switches in your smartphone behave differently depending on the current state of the device:
- When the phone is unlocked, pressing buttons leads to executing various functions.
- When the phone is locked, pressing any button leads to the unlock screen.
- When the phone's charge is low, pressing any button shows the charging screen.

Template pattern

- *Defer the exact steps of an algorithm to a subclass.*
- Pattern defines steps of an algorithm as methods in an interface while allowing subclasses to override some steps in the overall algorithm.
- Template Method design pattern defines the **skeleton** of an **algorithm** in the **superclass** but lets **subclasses override** specific steps of the algorithm without changing its structure.
- The *Template Method* defines a skeleton of an algorithm in an operation, and **defers** some steps to **subclasses**.

Template Method pattern

- Home builders use the *Template Method* when developing a new subdivision.
- A typical subdivision consists of a limited number of floor plans, with different variations available for each floor plan.
- Within a floor plan, the foundation, framing, plumbing, and wiring will be identical for each house.

Template Method Pattern Example

- In a report generation module, customer requirement is to generate different type of reports either HTMLReport, PDF report, excel report or in a flat file report where the report header, logo and footer keep constant.

Visitor Design pattern

- It is used when we have to perform *an operation* on a *group of similar kind of Objects*. With the help of visitor pattern, we can move the operational logic from the objects to another class.
- It is a way of *separating an algorithm* from an object structure on which it operates.
- A practical result of this separation is the ability to *add new operations* to existing object structures without modifying the structures.

Visitor Design Pattern

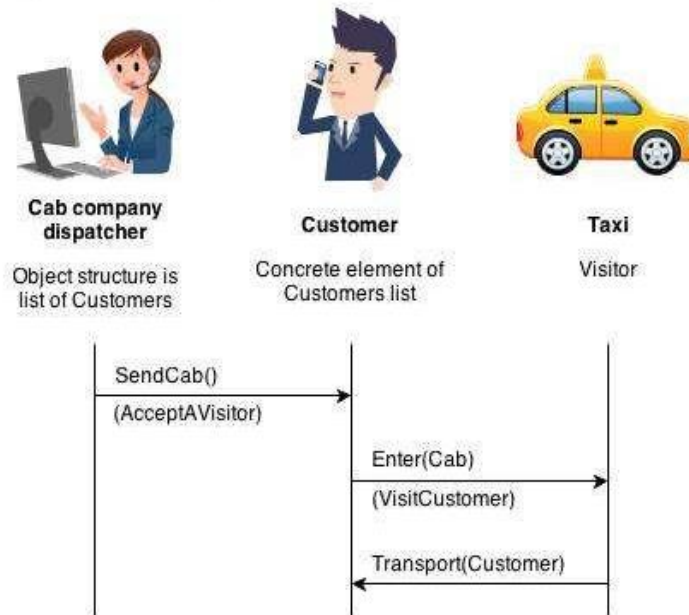
- Imagine an insurance agent who's eager to get new customers.
- He can visit every building in a neighborhood, trying to sell insurance to everyone he meets.
- Depending on the type of organization that occupies the building, he can offer *specialized* insurance policies:
- If it's a residential building, he sells medical insurance.
- If it's a bank, he sells theft insurance.
- If it's a coffee shop, he sells fire and theft insurance.

Visitor Design pattern

- Shopping in the supermarket is another common example, where the shopping cart is your set of *elements*.
- When you get to the checkout, the cashier acts as a *visitor*, taking the disparate set of elements (your shopping), some with prices and others that need to be weighed, in order to provide you with a total.

Visitor Design pattern

- This pattern can be observed in the operation of a taxi company.
- When a person calls a taxi company (accepting a visitor), the company dispatches a cab to the customer.
- Upon entering the taxi the customer, or **Visitor**, is no longer in control of his or her own transportation, the taxi (driver) is..



USEFUL LINKS

<https://refactoring.guru/design-patterns>

https://sourcemaking.com/design_patterns



CONTINUE....



ANY THING TO ASK

??